

Human-Machine Dialogue exam report

Marco Ragusa

Human-Machine Dialogue Class 2025

Department of Information Engineering and Computer Science

University of Trento

Trento, Italy

February 4, 2025

Contents

1	Introduction	2
1.1	Dialogue System Description	2
1.2	User Group	2
2	Conversation Design	2
2.1	Types of dialogues supported	2
2.2	Interaction properties	2
3	Conversation Model	3
3.1	Pipeline	3
3.2	Components	3
4	Evaluation	4
4.1	Intrinsic evaluation	4
4.2	Discussion	4
5	Extrinsic Evaluation	5
5.1	Participants and procedure	5
5.2	Results and discussion	5
6	Conclusion	5
A	List of intents and slots	6
A.1	Plan (<code>plan</code>)	6
A.2	Menu selection (<code>select_{menu}</code>)	6
A.3	Inspect a day (<code>inspect</code>)	6
A.4	Show week overview (<code>show_{week}</code>)	6
A.5	Refinement (<code>refine</code>)	6
A.6	Confirm/finalize (<code>confirm</code>)	6
A.7	Help (<code>help</code>)	7
A.8	Out-of-domain and typed control cases (<code>out_{ofdomain}</code>)	7
A	Human evaluation questionnaire	7

1 Introduction

1.1 Dialogue System Description. *Meal Kit Composer* is a task-oriented dialogue system that helps a user compose a 5-day dinner plan (Monday to Friday) from a fixed local recipe dataset. The interaction is structured as: (i) gather planning constraints, (ii) generate and present two alternative weekly menus that satisfy those constraints, (iii) allow the user to inspect meals and iteratively refine the plan (e.g., by substituting a day’s meal or updating dietary avoids), and (iv) finalize the plan by producing a consolidated shopping list with ingredient quantities scaled to the requested number of servings.

Supported exchanges. The system supports slot-based planning (`servings`, `time_limit`, `calorie_level`, `avoid_items`), menu proposal and selection (option #1 vs. #2), week-level visualization and per-day inspection, and refinement through day substitutions and avoid-list updates. Refinements follow a safe pattern: the system can suggest an alternative without mutating the plan, and commits changes only after an explicit confirmation.

System constraints. All meals are selected from a local JSON dataset (100 recipes) and the plan is defined over {Mon, Tue, Wed, Thu, Fri}. The system is hybrid: LLMs are used for NLU and DM; NLG is mostly template-based with an LLM fallback. Constraint checking, swaps/repairs, and shopping-list aggregation are implemented deterministically, while menu generation is implemented algorithmically (with controlled randomness for variety).

1.2 User Group. The target users are people who want a quick weekly dinner plan with minimal effort, including: (i) busy users optimizing for speed and convenience, (ii) users with dietary restrictions/allergens who need reliable exclusion of unwanted foods, and (iii) students or households seeking a lightweight planner that outputs a ready-to-shop ingredient list.

2 Conversation Design

2.1 Types of dialogues supported.

Happy path. The primary workflow is: *PLAN* → *PROPOSE TWO MENUS* → *SELECT MENU* → (*INSPECT/REFINE*) → *CONFIRM*. The system collects constraints one at a time, proposes two feasible weekly menus, and requires the user to select one menu before enabling plan-dependent operations (inspection, refinement, confirmation).

Iterative refinement. After a menu is selected, the system supports open-ended refinement loops: users can inspect any day multiple times, request one or more alternative meals, accept or refuse swap suggestions, and update `avoid_items`. Avoid updates trigger an automatic repair step: any day that violates the updated constraints is replaced with a feasible alternative.

2.2 Interaction properties.

Initiative and mixed initiative. During planning, the dialogue is slot-driven: the system actively requests missing constraint values one at a time until the plan is complete. Once a menu is active, the dialogue becomes mixed-initiative: users can jump between viewing the week, inspecting a day, requesting swaps, updating avoids, asking for help, and confirming. Phase-based gating ensures that actions remain coherent (e.g., confirmation and swaps are blocked until a menu is selected).

Engagement and transparency. The system provides two alternatives for the same constraints to promote exploration with low effort. It presents a compact week overview and a more detailed day view (steps and scaled ingredients). Potentially disruptive edits use suggest-then-confirm, which keeps the interaction safe and transparent: a suggestion shows what would change, and a separate confirmation turn commits it.

Grounding and robustness. Short acknowledgements are handled explicitly (treated as no-ops), reducing accidental state changes and fallbacks. Clarification questions are slot-targeted (one

missing value per turn). Invalid or out-of-scope requests trigger a safe fallback that does not corrupt state.

Confirmation patterns. Three confirmations are central: (i) a *menu-selection gate* before enabling edits; (ii) explicit yes/no confirmation for committing a suggested swap; and (iii) a “continue?” confirmation when the user expresses multiple requests in a single turn, ensuring one-action-per-turn execution without silently dropping intent.

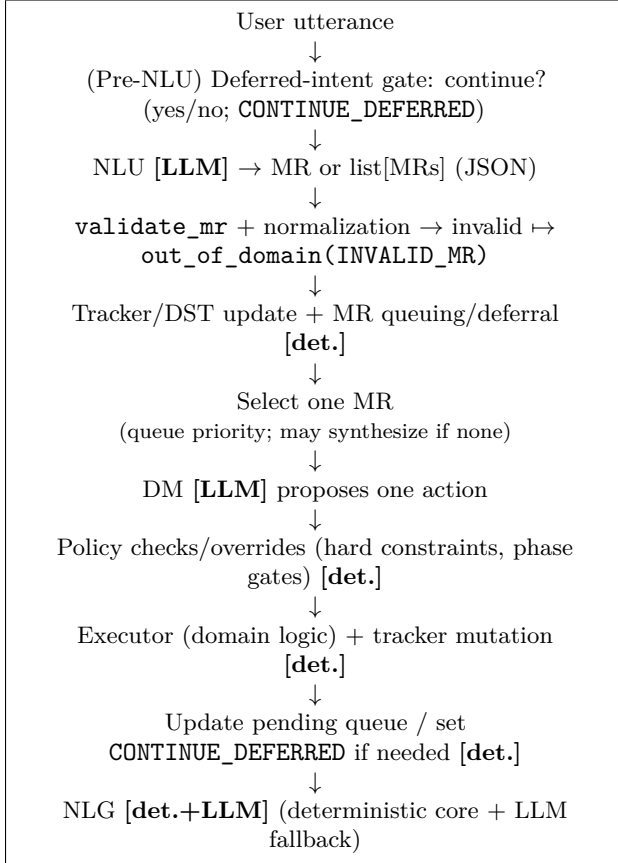


Figure 1: Meal Kit Composer pipeline (LLM vs deterministic stages).

3 Conversation Model

3.1 Pipeline. The system follows a hybrid LLM+rules pipeline (Fig. 1). Each user turn is mapped by an LLM based NLU into one or more JSON meaning representations (MRs). MRs are then validated and normalized to prevent malformed outputs from corrupting state. A tracker maintains the dialogue phase, constraints, menus,

and pending confirmations. A compact LLM based DM proposes one next action from a closed action set, while a deterministic policy layer enforces hard guardrails (slot prompting, phase gating, confirmation routing). Finally, deterministic domain logic executes the action (menu generation, swaps/repairs, shopping list) and an NLG module produces the user-facing response. To preserve predictability, the controller executes one MR per turn and defers additional requests behind an explicit “continue?” gate (CONTINUE_DEFERRED).

3.2 Components.

NLU and schema (nlu.py, intents_schema.py). The NLU is prompted to output JSON-only MRs and may return a list of MRs for multi-intent inputs. When a specific slot is being filled, the NLU uses an evidence gate: it only maps the user input to one of the allowed options when the text contains an explicit match; otherwise it labels the turn as out-of-domain (e.g., invalid answer) and triggers a reprompt instead of guessing. The schema layer validates and normalizes slots (e.g., day and option normalization) before they are applied to state.

State and control (support_classes.py, main.py). The tracker stores the planning constraints, the two generated menus, the selected active menu, and control variables such as `awaiting_slot`, deferred MRs, and pending confirmations (notably swap confirmation). The main controller enforces the one-action-per-turn discipline and implements the continue gate for deferred requests. A small rolling History of recent turns is also maintained and used only as context for LLM prompting in NLU/DM and for the LLM fallback in NLG; it does not participate in deterministic state updates or policy gating.

DM and policy (dm.py, policy.py). The DM produces exactly one action in a closed set given the selected MR and a compact tracker snapshot. The policy layer enforces hard rules independent of the DM, including deterministic slot prompting, menu-selection gating, and routing for confirmation-dependent operations.

Domain execution and NLG (`support_fn.py`, `nlg.py`). The deterministic execution layer filters infeasible recipes, generates menus, applies day swaps, triggers automatic repairs after avoid updates, and aggregates a servings-scaled shopping list. NLG is hybrid: structured outputs (menus, day views, reprompts, confirmations, shopping list) are rendered deterministically, with an LLM fallback for less structured cases, conditioned on an input bundle containing the tracker snapshot and execution payload (and, when useful, a short window of recent turns from History).

4 Evaluation

4.1 Intrinsic evaluation.

Data description & analysis. For NLU, a template-based string-injection dataset ($n=1500$, `nlu_injection_v1`) was used as the primary benchmark. For DM, next-best-action decisions were evaluated on an expanded scenario replay set ($n=232$ decision points, `scenarios_v1_big`). For NLG, a small gold set of action-to-utterance items ($n=30$) was used.

Description of intrinsic evaluation. NLU was evaluated by comparing predicted meaning representations against gold labels, reporting intent accuracy, slot micro precision/recall/F1, and exact match. Different base models were compared using the same production prompt (v3). In addition, a prompt ablation compared v2 vs. v3 with the model held fixed. DM was evaluated as a next-action classifier conditioned on tracker state and the selected meaning representation, reporting pre-policy accuracy and macro-F1 plus a per-action summary. Since the system includes deterministic guardrails, the policy override rate (how often the policy replaces the raw DM output) is also reported to quantify the contribution of constraints. NLG was evaluated with an action-leak constraint (whether internal action strings appear in user-facing text); overlap scores (BLEU/ROUGE) are reported only as supporting evidence.

Results of intrinsic evaluation. NLU results are summarized in Table 1 (injection set), including

the prompt ablation (v2 vs. v3). DM results are reported in Table 2 and Table 4 (pre-policy), together with the policy override rate to quantify how often guardrails corrected DM outputs. NLG results are summarized in Table 3; the action leak rate is used as the primary automatic check, and one qualitative failure example is included to show why overlap metrics alone do not guarantee semantic correctness.

4.2 Discussion. Overall, intrinsic results show that NLU performance depends on both the base model and prompt alignment, motivating the prompt ablation analysis. For DM, reporting pre-policy metrics is necessary because deterministic policy constraints can mask raw DM errors; the override rate makes this effect explicit. For NLG, constraint checks (e.g., action leak) are more reliable than surface overlap metrics, so overlap scores are treated as secondary and complemented with a qualitative error example.

Table 1: NLU intrinsic evaluation on injection set ($n = 1500$).

Model	Prompt	Acc	Prec.	Rec.	F1	EM
Llama3.1	v3	0.970	0.903	0.903	0.903	0.619
Qwen3	v3	0.837	0.914	0.791	0.848	0.669
Qwen3	v2	0.833	0.887	0.768	0.823	0.586

Table 2: DM intrinsic evaluation (llama31, $n = 232$): overall metrics (pre-policy) and policy override rate.

Metric	Value
Pre-policy accuracy	0.543
Pre-policy macro-F1	0.556
Policy override rate	0.457 (106/232)

Table 3: NLG intrinsic evaluation (llama31, $n = 30$).

Metric	Value
BLEU	0.724
ROUGE-1 (F1)	0.808
ROUGE-2 (F1)	0.771
ROUGE-L (F1)	0.803
Action leak rate	0.000

Table 4: DM intrinsic evaluation (llama31): per-action F1 and support (pre-policy), $n = 232$.

Action	F1	Support
request_info	0.547	64
propose_menus	0.000	48
set_active_menu	0.476	48
show_day	0.979	24
confirm_plan	0.000	16
show_week	1.000	8
suggest_swap_day	1.000	8
swap_day	1.000	8
update_avoid	0.000	8

5 Extrinsic Evaluation

A small-scale human (extrinsic) evaluation was conducted on the complete *Meal Kit Composer* pipeline, i.e., the end-to-end user experience when the NLU, DM, and NLG components are integrated. The evaluated workflow corresponds to the intended system flow (plan constraints $\rightarrow$ generate two menus $\rightarrow$ select active menu $\rightarrow$ optional inspect/refine $\rightarrow$ confirm and obtain shopping list).

5.1 Participants and procedure. Responses were collected from 8 participants (P01–P08), excluding the initial test entry (P00). Each participant interacted freely with the system and then completed a post-task questionnaire with seven questions (Q1–Q7), each rated on a 1–5 scale (higher is better). The questionnaire questions are reported in the appendix.

5.2 Results and discussion. Table 5 summarizes the post-task ratings. Overall, participants reported a moderately positive experience (overall mean 3.66 ± 0.63). The strongest aspects were constraint satisfaction (Q1, 4.50 ± 0.53) and interaction guidance (Q4, 4.25 ± 1.16), suggesting that users generally perceived the system as following their preferences and presenting a clear next step in the workflow (e.g., selecting a menu, refining, confirming). Perceived response quality was also positive (Q3, 4.00 ± 0.93), indicating that the NLG outputs were mostly clear and coherent. Refinement operations were rated slightly lower but still above neutral (Q5, 3.86 ± 1.21); the higher variance here suggests that refinement behavior was not equally reliable across participants or scenarios. The main

weaknesses concern trust and clarification. Trust in the final output was the lowest-rated item (Q6, 2.75 ± 1.28), and the system’s ability to proactively ask for missing/unclear information was only neutral (Q2, 3.00 ± 0.76). Together, these results indicate that while the system is usable and produces plans aligned with stated constraints, participants are less confident in adopting the output without additional verification. This points to improving clarification strategies (e.g., more robust detection of underspecified constraints) and increasing transparency/justification of key decisions (e.g., why a recipe was selected or rejected) to better support user trust. Finally, reuse intention is slightly above neutral (Q7, 3.25 ± 0.71), consistent with a system that is promising but would benefit from reliability and trust-oriented improvements.

Item	Mean	SD
Q1	4.50	0.53
Q2	3.00	0.76
Q3	4.00	0.93
Q4	4.25	1.16
Q5	3.86	1.21
Q6	2.75	1.28
Q7	3.25	0.71
Overall (avg. Q1–Q7)	3.66	0.63

Table 5: Human ratings (P01–P08) on a 5-point scale for questionnaire items Q1–Q7 (mean and standard deviation).

6 Conclusion

This report presented *Meal Kit Composer*, a task oriented dialogue system that combines LLM based NLU and DM (and an LLM fallback for NLG) with deterministic validation, policy guardrails, and domain execution for menu generation, plan edits, and shopping list aggregation. The intrinsic evaluation shows strong NLU performance on the injection benchmark (best setting: Llama3.1 with the v3 prompt, intent accuracy 0.97). DM performance is lower before applying the policy layer (accuracy 0.543), and many errors are corrected by deterministic constraints (policy override rate 0.457). This indicates that hard workflow rules are important to keep the interaction reliable. In the human evaluation (8 participants), users rated constraint satisfaction and guidance positively (Q1=4.50, Q4=4.25). Trust in the final output was the weakest item (Q6=2.75), which suggests that the system is us-

able but still needs improvements to better support user confidence. Future work will focus on improving clarification when inputs are underspecified, reducing DM errors on critical actions, and increasing transparency when the system selects or repairs meals.

References

- [1] AI@Meta. *Meta-Llama-3.1-8B-Instruct (model card)*. 2024. <https://huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct>
- [2] Qwen Team. *Qwen3 Technical Report*. arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>
- [3] T. Wolf et al. *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP 2020: System Demonstrations, 2020. <https://aclanthology.org/2020.emnlp-demos.6/>
- [4] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS 2019. <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [5] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. *BLEU: a Method for Automatic Evaluation of Machine Translation*. ACL 2002. <https://aclanthology.org/P02-1040/>
- [6] C.-Y. Lin. *ROUGE: A Package for Automatic Evaluation of Summaries*. Text Summarization Branches Out, 2004. <https://aclanthology.org/W04-1013/>
- [7] R. Likert. *A Technique for the Measurement of Attitudes*. Archives of Psychology, no. 140, 1932. <https://kansalliskirjasto.finna.fi/Record/fikka.754188>

A List of intents and slots

The NLU outputs meaning representations (MRs) in the flat format `{intent: ..., slots: {...}}`. Some user turns may yield a list of MRs (multi-intent), but each MR conforms to the intent/slot inventory below.

A.1 Plan (plan). Planning constraints used for menu generation.

- `servings`: integer in `{1,2,3,4,5,6}`.
- `time_limit`: `FAST` | `NORMAL`.
- `calorie_level`: `LOW` | `MED` | `HIGH`.
- `avoid_items`: list of strings (possibly empty []).
 - Controlled allergy-style tags supported by the system: `nuts`, `dairy`, `gluten`, `soy`, `egg`, `sesame`, `fish`, `shellfish`, `meat`.
 - Additionally, arbitrary free-text ingredient/food keywords are accepted (e.g., “mushrooms”, “broccoli”).

A.2 Menu selection (select_menu). Choose which proposed weekly menu becomes active.

- `menu_id`: integer `1` | `2`.

A.3 Inspect a day (inspect). Request details for a specific weekday in the current plan.

- `target_day`: `Mon` | `Tue` | `Wed` | `Thu` | `Fri`.

A.4 Show week overview (show_week). Display the full Mon–Fri plan.

- (no slots)

A.5 Refinement (refine). All edits to an active plan are expressed via `refine`, with a required `refine_type`.

- `refine_type`: `SWAP_DAY` | `ADD_AVOID_ITEM` | `REMOVE_AVOID_ITEM`.
- `mode`: `SUGGEST` | `COMMIT` (optional; used for swap requests).
- `target_day`: weekday (only meaningful/required for `SWAP_DAY`).
- `value`: required; interpretation depends on `refine_type`:
 - If `refine_type=SWAP_DAY`:
 - * `target_day` is required (`Mon–Fri`).
 - * `value` must be `BEST_FIT` (the system’s deterministic swap strategy).
 - If `refine_type=ADD_AVOID_ITEM` or `REMOVE_AVOID_ITEM`:
 - * `value` is a non-empty string (controlled tag or free-text keyword).
 - * `target_day` must be omitted/null (the update applies globally).

A.6 Confirm/finalize (confirm). Finalize the selected plan and produce the shopping list.

- (no slots)

A.7 Help (help). Ask what the system can do, optionally scoped to an intent/slot.

- **intent:** required; one of `plan`, `select_menu`, `inspect`, `refine`, `confirm`, `show_week`, `help`, `out_of_domain`.
- **slot:** optional; if provided, must be a valid slot name for the chosen intent (e.g., for `plan`: `servings`, `time_limit`, `calorie_level`, `avoid_items`).

A.8 Out-of-domain and typed control cases (`out_of_domain`). Safe fallback intent, also used for typed dialogue-control cases.

- **ood_type:** optional; observed system types include: `ACK` (acknowledgement/no-op), `INVALID_ANSWER` (invalid reply while a specific slot is awaited), `REFUSE_PENDING` (explicit refusal in a pending confirmation context), `INVALID_MR` (schema-invalid MR recovered safely).

A Human evaluation questionnaire

After completing the assigned scenario(s), participants answered the following seven items on a 5-point Likert scale (1 = strongly disagree, 5 = strongly agree):

- **Q1.** The system followed my constraints (servings, time, calories, avoid items).
- **Q2.** The system asked for missing information or clarifications when something was unclear.
- **Q3.** The system’s responses were clear and coherent (no contradictions or irrelevant details).
- **Q4.** It was easy to understand what to do next (e.g., select menu option, refine, confirm).
- **Q5.** Refinements worked as I expected (swap day / update avoid items / repair).
- **Q6.** I trust the final output enough to use it for planning meals/shopping.
- **Q7.** I would consider using the system again.